

FIAP - Faculdade de Informática e Administração Paulista

Pós-Graduação em Arquitetura e Desenvolvimento Java

Jefferson Ricardo dos Santos

RM371825

John Pablo da Silva Gomes

RM372387

**SISTEMA HOSPITALAR DE AGENDAMENTO DE CONSULTAS:
MICROSSERVIÇOS COM MENSAGERIA, JWT E GRAPHQL**

Tech Challenge — Fase 3

São Paulo

2026

DESCRIÇÃO DO PROJETO

Sistema hospitalar para agendamento de consultas médicas, construído com uma arquitetura de **microsserviços** e comunicação assíncrona. O sistema permite que médicos e enfermeiros criem, editem e gerenciem consultas; pacientes visualizam e cancelam as próprias consultas. Cada operação em uma consulta dispara um evento assíncrono que alimenta um serviço de notificações (lembretes) e um serviço de histórico (consultável via GraphQL).

São três microsserviços independentes, comunicando-se via RabbitMQ, com segurança baseada em JWT e seguindo os princípios da Clean Architecture (domain → application → infra).

Repositório:

<https://github.com/jeffesa/12ADJT-fase-3>

ARQUITETURA

Visão geral (microsserviços + mensageria)

O fluxo de ponta a ponta funciona assim:

1. O **cliente** faz uma requisição REST autenticada (JWT) ao **scheduling-service** (porta 8081) — por exemplo, criar uma consulta.
2. O **scheduling-service** persiste no seu banco PostgreSQL e **publica um evento** (JSON) na exchange do **RabbitMQ**.
3. O **RabbitMQ** (exchange `appointment.exchange`, tipo *topic*) roteia o evento para duas filas ao mesmo tempo.
4. O **notification-service** (porta 8082) consome o evento e registra a notificação.
5. O **history-service** (porta 8083) consome o mesmo evento e persiste o histórico, que fica disponível via GraphQL.

Resumo do encaminhamento:

cliente → (REST + JWT) → scheduling-service → (evento) → RabbitMQ → notification-service e history-service

Papel de cada serviço

Serviço	Porta	Banco	Responsabilidade
scheduling-service	8081	PostgreSQL (5433)	Núcleo: autenticação JWT, CRUD de consultas, controle de acesso por role/ownership, scheduler de lembretes e publicação de eventos
notification-service	8082	H2 (em memória)	Consome eventos, gera/registra notificações (incluindo lembretes), API REST de consulta, retry + Dead Letter Queue
history-service	8083	PostgreSQL (5434)	Consome eventos, persiste o histórico das consultas, expõe consultas via GraphQL
RabbitMQ	5672 / 15672	—	Message broker (exchange <code>appointment.exchange</code> , tipo <i>topic</i>) que desacopla o produtor dos consumidores

Clean Architecture (por serviço)

Cada microsserviço segue as três camadas com dependência de fora para dentro:

- **domain/** — Entidades puras (POJOs), interfaces de gateway, eventos e exceções de domínio. Zero framework.
- **application/** — Use cases que orquestram os fluxos de negócio. Depende apenas do domain.
- **infra/** — Controllers REST/GraphQL, DTOs, entidades JPA, repositories, configuração Spring, messaging (RabbitMQ) e segurança.

Comunicação assíncrona (RabbitMQ)

- **Exchange:** appointment.exchange (tipo *topic*)
- **Routing keys:** appointment.created, appointment.updated, appointment.reminder
- **Filas:** appointment.notification.queue e appointment.history.queue, cada uma com sua DLQ (.dlq)
- **Resiliência:** 3 tentativas com backoff exponencial; mensagens que falham vão para a DLQ (TTL 24h)

O produtor (scheduling) publica o evento e não conhece os consumidores. O RabbitMQ entrega para as filas conforme a routing key. Se um consumidor estiver fora do ar, a mensagem aguarda na fila até ser processada (desacoplamento e resiliência).

Scheduler de lembretes

O scheduling-service possui um agendador (@Scheduled) que roda periodicamente (a cada 15 min por padrão), busca consultas nas próximas 24h ainda não lembradas e publica um evento appointment.reminder. O notification-service consome e registra o lembrete. Uma flag reminderSent na consulta evita envio duplicado.

TECNOLOGIAS UTILIZADAS

Categoria	Tecnologia	Uso
Linguagem	Java 17	Linguagem principal
Framework	Spring Boot 3.2.x	Base dos três serviços
Build	Maven (multi-módulo)	Compilação e testes
Segurança	Spring Security + JWT (jjwt 0.12.5)	Autenticação/autorização
Mensageria	RabbitMQ (Spring AMQP) + DLQ	Comunicação assíncrona
Persistência	Spring Data JPA + PostgreSQL 15 (prod) / H2 (dev/test)	Bancos por serviço
API	REST (scheduling, notification) + GraphQL (history)	Interfaces
Documentação	SpringDoc OpenAPI / Swagger	Docs interativas
Testes	JUnit 5 + Mockito	Unitários e integração
Cobertura	JaCoCo 0.8.13 ($\geq 80\%$ por módulo)	Qualidade
Qualidade	SonarCloud	Análise estática
Containerização	Docker + Docker Compose	Orquestração local
CI/CD	GitHub Actions	Build, testes e validação

SERVIÇOS, PORTAS E ENDEREÇOS

O ambiente sobe seis containers em uma rede Docker dedicada (fase3net, driver bridge). Cada serviço tem um endereço **interno** (usado na comunicação entre containers, pelo nome do serviço) e um endereço **externo** (mapeado para o localhost da máquina, usado por você e pelas ferramentas).

Tabela de serviços

Serviço (container)	Porta externa (host)	Porta interna (container)	Endereço externo (você acessa)	Endereço interno (entre containers)	Para que serve
fase3-scheduling-service	8081	8081	http://localhost:8081	scheduling-service:8081	API REST: auth (JWT), CRUD de consultas, scheduler de lembretes
fase3-notification-service	8082	8082	http://localhost:8082	notification-service:8082	Consome eventos, registra notificações, API REST de notificações
fase3-history-service	8083	8083	http://localhost:8083	history-service:8083	Consome eventos, persiste histórico, API GraphQL
fase3-postgres-scheduling	5433	5432	localhost:5433	postgres-scheduling:5432	Banco do scheduling (scheduling_db)
fase3-postgres-history	5434	5432	localhost:5434	postgres-history:5432	Banco do history (history_db)
fase3-rabbitmq	5672 / 15672	5672 / 15672	localhost:5672 (AMQP) /	rabbitmq:5672	Message broker +

Serviço (container)	Porta externa (host)	Porta interna (container)	Endereço externo (você acessa)	Endereço interno (entre containers)	Para que serve
			http://localhost:15672 (UI)		console de gestão

Por que porta externa difere da interna nos Postgres?

Ambos os bancos usam a porta 5432 dentro do container, mas seriam conflitantes no host. Por isso são mapeados para 5433 (scheduling) e 5434 (history) na sua máquina. Entre containers, eles continuam se comunicando pela 5432 usando o nome do serviço (ex.: postgres-scheduling:5432).

Comunicação interna por nome:

Dentro da rede fase3net, os serviços se enxergam pelo nome do container/serviço, não por IP fixo. Ex.: o scheduling conecta ao banco via `jdbc:postgresql://postgres-scheduling:5432/scheduling_db` e ao broker via `rabbitmq:5672`. O Docker resolve esses nomes para os IPs internos automaticamente.

RabbitMQ — acesso ao console de gestão

Item	Valor
Console (UI web)	http://localhost:15672
Porta AMQP (mensageria)	5672
Usuário	guest
Senha	guest

No console, a aba **Queues and Streams** lista as filas (`appointment.notification.queue`, `appointment.history.queue` e as respectivas `.dlq`); a aba **Exchanges** mostra a `appointment.exchange` (topic).

Endereços úteis (após subir a aplicação)

Recurso	URL
Scheduling – Swagger UI	http://localhost:8081/swagger-ui.html
Scheduling – Health	http://localhost:8081/actuator/health
Notification – Swagger UI	http://localhost:8082/swagger-ui.html
Notification – Health	http://localhost:8082/actuator/health
History – GraphiQL (UI de queries)	http://localhost:8083/graphiql
History – Health	http://localhost:8083/actuator/health
RabbitMQ – Console	http://localhost:15672 (guest/guest)

COMO RODAR A APLICAÇÃO (run.sh)

O projeto inclui o script run.sh na raiz, que centraliza as operações mais comuns. Ele pode ser usado de forma **interativa** (menu) ou **direta** (passando a ação como argumento).

Pré-requisitos

- **Docker e Docker Compose** em execução (no macOS, pode ser via Docker Desktop ou Colima)
- **Java 17+** e **Maven** — necessários apenas para rodar os testes localmente (não para o Docker)
- **Node.js/npm** — necessário apenas para rodar a collection via Newman

Uso interativo

./run.sh

Exibe o menu abaixo:

Tech Challenge Fase 3 – Sistema Hospitalar
--

- 1) Subir tudo (Docker Compose up --build)
- 2) Parar containers (Docker Compose down)
- 3) Rodar testes (mvn clean verify)
- 4) Limpar bancos (Docker Compose down -v)
- 5) Liberar portas (8081, 8082, 8083)
- 6) Health check dos serviços
- 7) Ver logs (Docker Compose logs -f)
- 8) Rodar collection (Newman)
- 0) Sair

Uso direto

`./run.sh <ação>`

Opções do menu / ações disponíveis

Opção	Ação (argumento)	O que faz
1	<code>docker / up</code>	Sobe toda a aplicação via Docker Compose (<code>up --build -d</code>). Garante o daemon Docker ativo (inicia o Colima se necessário) e imprime os endereços úteis (Swagger, health, GraphiQL, RabbitMQ).
2	<code>stop / down</code>	Para e remove os containers (<code>docker-compose down</code>).
3	<code>tests / test</code>	Roda os testes com <code>mvn clean verify</code> no profile <code>test</code> (H2 em memória, sem RabbitMQ). Valida a cobertura JaCoCo.
4	<code>reset-db</code>	Remove os volumes/bancos (<code>docker-compose down -v</code>). Use para começar com o banco limpo.
5	<code>kill</code>	Libera as portas 8081, 8082 e 8083 (encerra processos que estiverem ocupando).
6	<code>health</code>	Verifica o <code>/actuator/health</code> dos três serviços e reporta UP/indisponível.
7	<code>logs</code>	Acompanha os logs de todos os containers em tempo real (<code>docker-compose logs -f</code>).
8	<code>newman / collection</code>	Roda a collection de testes de API (Newman) contra o ambiente local. Requer os serviços no ar e Node.js instalado.
0	<code>-</code>	Sai do menu.

Fluxo recomendado de primeira execução

1. Subir tudo

`./run.sh docker`

2. Conferir a saúde dos serviços

`./run.sh health`

3. (opcional) Rodar a collection de API

`./run.sh newman`

4. Ao terminar, parar os containers

`./run.sh stop`

Alternativa sem o script

`docker-compose up --build -d` `# subir`

`docker-compose logs -f` `# logs`

`docker-compose down` `# parar`

`docker-compose down -v` `# parar e limpar volumes`

ENDPOINTS DA API

Documentação detalhada (request/response, exemplos de erro, DTOs) em [docs/API.md](#).
Resumo abaixo.

Autenticação (scheduling-service — público)

Método	Endpoint	Descrição	Status
POST	/api/v1/auth/register	Registrar usuário (retorna JWT)	201, 400, 422
POST	/api/v1/auth/login	Autenticar (retorna JWT)	200, 422

Roles: DOCTOR, NURSE, PATIENT. O token é enviado nas requisições protegidas via header Authorization: Bearer <token>.

Consultas (scheduling-service — requer JWT)

Método	Endpoint	Descrição	Roles	Status
POST	/api/v1/appointments	Criar consulta	DOCTOR, NURSE	201, 400, 403, 404, 422
PUT	/api/v1/appointments/{id}	Editar consulta	DOCTOR, NURSE	200, 403, 404, 422
PATCH	/api/v1/appointments/{id}/confirm	Confirmar (SCHEDULED→CONFIRMED)	DOCTOR, NURSE	200, 403, 404, 422
PATCH	/api/v1/appointments/{id}/complete	Concluir (CONFIRMED→COMPLETED)	DOCTOR, NURSE	200, 403, 404, 422
PATCH	/api/v1/appointments/{id}/cancel	Cancelar	DOCTOR, NURSE, PATIENT (própria)	200, 403, 404, 422
GET	/api/v1/appointments/{id}	Buscar por ID	autenticado (ownership)	200, 403, 404
GET	/api/v1/appointments	Listar todas (filtro ?status=)	DOCTOR, NURSE	200, 400, 403
GET	/api/v1/appointments/patient/{patientId}	Consultas de um paciente	DOCTOR, NURSE ou o próprio	200, 403

Método	Endpoint	Descrição	Roles	Status
GET	/api/v1/appointments/doctor/{doctorId}	Consultas de um médico	DOCTOR, NURSE	200, 403
GET	/api/v1/appointments/upcoming	Consultas futuras (por role)	autenticado	200

Usuários (scheduling-service — requer JWT)

Método	Endpoint	Descrição	Roles	Status
GET	/api/v1/users	Listar usuários (filtro ?role=)	DOCTOR, NURSE	200, 400, 403
GET	/api/v1/users/{id}	Buscar por ID	autenticado (PATIENT só o próprio)	200, 403, 404

Notificações (notification-service — requer JWT)

Método	Endpoint	Descrição	Status
GET	/api/v1/notifications	Listar notificações (filtro ?type=)	200, 422

Tipos: APPOINTMENT_CREATED, APPOINTMENT_UPDATED, APPOINTMENT_REMINDER.

Histórico (history-service — GraphQL, requer JWT)

Endpoint: POST /graphql — UI de testes em GET /graphiql.

Query / Mutation	Descrição
appointmentsByPatient(patientId)	Histórico de um paciente
appointmentsByDoctor(doctorId)	Histórico de um médico
upcomingAppointments(patientId)	Consultas futuras de um paciente
appointmentHistory(id)	Um registro específico
allAppointmentHistories	Todos os registros (DOCTOR/NURSE)

Query / Mutation	Descrição
saveAppointmentHistory(input)	Inserção manual/administrativa

COLLECTION DE API (chamadas dos microserviços)

O repositório inclui uma collection única que **demonstra e testa as chamadas de todos os microserviços** da aplicação, em docs/api-collection/fase3-hospital.postman_collection.json. É compatível com **Postman**, **Bruno** e **Newman** (schema Postman Collection v2.1.0) e traz as variáveis (URLs dos três serviços, token JWT, IDs) embutidas na própria collection — não há arquivo de environment separado.

Organização (por microserviço)

As chamadas ficam na pasta local (ambiente Docker Compose), agrupadas por serviço/domínio:

Pasta	Serviço	O que cobre
Auth	scheduling-service (8081)	Registro (DOCTOR, PATIENT, NURSE) e login; casos de erro (credenciais inválidas, email duplicado, body/role inválidos)
Scheduling	scheduling-service (8081)	CRUD de consultas, ciclo de status (confirmar/concluir/cancelar), filtros por status e casos de erro (401/403/404/422)
Users	scheduling-service (8081)	Listagem de usuários com filtros por role e busca por ID; casos de erro (400/403/404)
Notification	notification-service (8082)	Consulta das notificações geradas pelos eventos, com filtro por tipo; caso de erro (tipo inválido, sem token)
History	history-service (8083)	Queries GraphQL do histórico (por paciente, por médico, futuras, por ID, todas); caso de erro sem token

Cada requisição possui **scripts de teste** (assertions) que validam o status HTTP e o corpo da resposta, cobrindo tanto cenários de sucesso quanto de erro.

Fluxo entre os serviços demonstrado pela collection

A collection evidencia a comunicação assíncrona entre os microserviços: ao criar/editar uma consulta na pasta **Scheduling** (scheduling-service), o evento é publicado no RabbitMQ e consumido pelos outros dois serviços — o que pode ser conferido em seguida nas pastas **Notification** (notificação gerada) e **History** (registro persistido, consultável via GraphQL). Assim, o mesmo appointmentId criado no scheduling aparece nas notificações e no histórico.

Como importar no Bruno

1. Bruno > **Import Collection** > Escolha **Postman Collection** e selecione fase3-hospital.postman_collection.json.
2. Abra a pasta local e rode **"Selecionar ambiente local"** primeiro (aponta as variáveis para localhost:8081/8082/8083 e limpa o token).
3. Rode a pasta Auth (o **Register DOCTOR** e o **Login DOCTOR** salvam o token; **Register PATIENT** salva o patientId).
4. Rode Scheduling, Users, Notification, History.

O auto-login (script de pré-request no nível da collection) faz login automaticamente se o token estiver vazio e já houver credenciais de DOCTOR salvas, então requests isolados também funcionam.

Como usar no Postman

Import > selecione o arquivo. Rode local > Selecionar ambiente local e depois as pastas na ordem Auth → Scheduling → Users → Notification → History.

Os serviços precisam estar no ar (./run.sh docker) antes de rodar a collection.

Variáveis (embutidas na collection)

Variável	Descrição
localScheduling / localNotification / localHistory	URLs locais (8081 / 8082 / 8083)
baseUrl / notificationUrl / historyUrl	URLs ativas, definidas pelo request "Selecionar ambiente local"
token	JWT do DOCTOR (usado nos requests autenticados)
patientToken / nurseToken	JWT de PATIENT e NURSE (cenários de ownership e role)
doctorId / patientId / nurseId	IDs dos usuários registrados
appointmentId	ID da consulta criada
appointmentDateTime	data/hora futura gerada no pré-request

Cenários

Sucesso: registro (DOCTOR, PATIENT, NURSE) e login; criar, buscar, atualizar, listar e cancelar consulta; confirmar e concluir (ciclo de status); listar por paciente e por médico; consultas futuras; listar usuários (com filtros de role); listar notificações (com e sem filtro por tipo); queries GraphQL (appointmentsByPatient, appointmentsByDoctor, allAppointmentHistories, appointmentHistory por id, upcomingAppointments).

Erro: login com senha inválida (422); registro com email duplicado (422), body inválido (400) ou role inválida (400); criar consulta sem token (401); listar com token inválido (401); buscar consulta inexistente (404); transição de estado inválida (422); acesso sem permissão por role/ownership (403); notificação com tipo inválido (422) e sem token (401); GraphQL sem token (401).

notification-service e history-service são populados de forma assíncrona pelos eventos RabbitMQ publicados pelo scheduling ao criar/atualizar consultas.

REGRAS DE NEGÓCIO

1. **Email único:** não podem existir dois usuários com o mesmo email.
2. **Validação de senha:** mínimo de 6 caracteres.
3. **Data no futuro:** a data/hora da consulta deve ser futura na criação e na edição.
4. **Criação/edição restrita:** apenas DOCTOR e NURSE criam e editam consultas.
5. **Ownership do médico:** um médico só edita/confirma/conclui as próprias consultas; enfermeiro tem alcance amplo.
6. **Ownership do paciente:** o paciente só visualiza e cancela as próprias consultas.
7. **Máquina de estados da consulta:** transições válidas são SCHEDULED → CONFIRMED → COMPLETED; o cancelamento é permitido a partir de SCHEDULED ou CONFIRMED. Consultas CANCELLED ou COMPLETED não podem mais ser alteradas.
8. **Lembrete único:** o scheduler marca reminderSent = true após enviar, evitando lembretes duplicados.
9. **Eventos assíncronos:** criação e edição de consultas publicam eventos que alimentam notificação e histórico.

SEGURANÇA (JWT)

- O token JWT é emitido **apenas** pelo scheduling-service (nos endpoints register e login). Os demais serviços apenas **validam** o token.
- Sessão *stateless*: nenhuma sessão é mantida no servidor.
- Autorização por role (DOCTOR, NURSE, PATIENT) e por ownership (validada nos use cases).
- Convenção HTTP: **401** para não autenticado (sem token, inválido ou expirado); **403** para autenticado sem permissão (role/ownership).

TRATAMENTO DE ERROS

Os serviços REST seguem o padrão **RFC 7807 ProblemDetail** (application/problem+json), com os campos type, title, status, detail, instance e timestamp.

Código	Quando ocorre
400 Bad Request	Falha de validação de campos; valor inválido no corpo (ex.: enum inexistente)
401 Unauthorized	Sem token, token inválido ou expirado
403 Forbidden	Autenticado, mas sem permissão (role/ownership)
404 Not Found	Recurso inexistente
422 Unprocessable Entity	Regra de negócio violada (ex.: email já cadastrado, transição de estado inválida)
500 Internal Server Error	Erro inesperado

No GraphQL, os erros vêm no array errors com extensions.code e extensions.status (não usam ProblemDetail).

TESTES E QUALIDADE

- **Testes automatizados** (unitários + integração + controller), executados com mvn clean verify no profile test (H2 em memória, sem dependência de RabbitMQ/PostgreSQL).
- **Cobertura $\geq 80\%$** por módulo, validada pelo **JaCoCo 0.8.13** (o build falha se a cobertura ficar abaixo do limite).
- **SonarCloud** integrado ao CI para análise estática (organização jeffesa, projeto jeffesa_12ADJT-fase-3).
- **Collection de API** (Postman/Bruno/Newman) em docs/api-collection/, cobrindo casos de sucesso e de erro dos três serviços (auth, consultas com todos os status, usuários, notificações e GraphQL), validável com ./run.sh newman.

Como rodar os testes

```
# todos os módulos (profile test, H2)
mvn clean verify -Dspring.profiles.active=test
```

```
# ou via script
./run.sh tests
```

O relatório de cobertura de cada módulo é gerado em
<módulo>/target/site/jacoco/index.html.

CI/CD

Pipeline em **GitHub Actions**, disparado em pull requests com base na branch develop:

- **Build & Test** — compila os três módulos, roda os testes e verifica a cobertura.
- **Branch Naming Convention** — valida o prefixo da branch (feature/, bugfix/, hotfix/, docs/, etc.).
- **SonarCloud** — análise de qualidade (executada quando o token está configurado).

O fluxo de trabalho segue o GitFlow: branches de feature//bugfix/ a partir de develop, revisão via pull request, e sincronização de main após o merge.

DECISÕES TÉCNICAS

Decisão	Justificativa
Microserviços	Separação de responsabilidades e escalabilidade independente por domínio
Comunicação assíncrona (RabbitMQ)	Desacoplamento entre produtor e consumidores; resiliência (a mensagem aguarda na fila se o consumidor cair)
Dead Letter Queue + retry	Não perder mensagens que falham; permitir reprocessamento e inspeção
Clean Architecture	Isolar regras de negócio da infraestrutura e do framework
JWT stateless	Autenticação sem estado, adequada a sistemas distribuídos
Banco por serviço (PostgreSQL)	Cada microserviço é dono dos seus dados
H2 em dev/test, PostgreSQL em prod	Agilidade nos testes, robustez em produção
GraphQL no history	Consultas flexíveis do histórico, expondo só o que o cliente pedir
Scheduler com flag reminderSent	Enviar lembrete uma única vez por consulta, sem duplicidade
ProblemDetail (RFC 7807)	Padrão internacional para respostas de erro
Docker Compose	Subir todo o ecossistema (3 serviços + 2 bancos + broker) com um comando

ESTRUTURA DO REPOSITÓRIO

Caminho	Conteúdo
scheduling-service/	Microserviço REST: autenticação, consultas e scheduler de lembretes
notification-service/	Microserviço consumidor: notificações + Dead Letter Queue
history-service/	Microserviço consumidor: histórico + API GraphQL
docs/API.md	Documentação detalhada dos endpoints
docs/api-collection/	Collection Postman/Bruno + execução via Newman
docs/Tech-Challenge-Fase3-Microserviços.md	Este documento
docker-compose.yml	Orquestração dos 6 containers
run.sh	Script runner (subir, testar, logs, etc.)
README.md	Visão geral e instruções do projeto

fase-3/

- └─ scheduling-service/ (REST: auth + consultas + scheduler)
- └─ notification-service/ (consumer + notificações + DLQ)
- └─ history-service/ (consumer + GraphQL)
- └─ docs/
 - └─ API.md (documentação detalhada dos endpoints)
 - └─ api-collection/ (collection Postman/Bruno + Newman)
 - └─ Tech-Challenge-Fase3-Relatorio.md (este documento)
- └─ docker-compose.yml (orquestração dos 6 containers)
- └─ run.sh (script runner)
- └─ README.md